

ПОЛНЫЕ И РАЗВЕРНУТЫЕ ОТВЕТЫ НА ЭКЗАМЕНАЦИОННЫЕ БИЛЕТЫ

Курс: Разработка приложений средствами Python / Проектирование ПО (ИСП)

Билет №1

1. История создания языка Python. Сферы применения:

Язык программирования Python был разработан в конце 1980-х годов сотрудником голландского института CWI Гвидо ван Россумом. Он задумывался как преемник языка ABC, способный взаимодействовать с операционной системой Атомеба и обладающий исключительной читаемостью кода. Первая официальная версия 0.9.0 была опубликована в 1991 году. Название произошло не от рептилии, а от любимого телешоу создателя — «Летающий цирк Монти Пайтона».

Важнейшими вехами стали: выпуск Python 2.0 в 2000 году (введение списковых включений, сборщика мусора) и релиз Python 3.0 в 2008 году. Переход на ветку 3.x исправил многие системные недостатки архитектуры, однако сознательно нарушил обратную совместимость с кодом 2.x, что вызвало длительный транзит сообщества.

Сегодня Python — один из самых востребованных языков. Основные сферы:

- **Data Science и ИИ:** де-факто стандарт индустрии благодаря экосистеме библиотек (NumPy, Pandas, Scikit-Learn, TensorFlow, PyTorch).
- **Веб-разработка:** создание бэкенда на базе мощных фреймворков Django, Flask, FastAPI.
- **Автоматизация и системное администрирование:** написание служебных скриптов, парсеров данных (BeautifulSoup, Scrapy) и автоматизация тестирования (PyTest, Selenium).

2. Термин API. Возможности применения API:

API (Application Programming Interface) — это интерфейс программирования приложений, представляющий собой набор готовых методов, классов, функций, структур и протоколов, предоставляемых программой или операционной системой для использования во внешних программных продуктах. API абстрагирует сложную внутреннюю логику сервиса, давая разработчикам простой инструмент для интеграции.

Возможности применения API крайне обширны:

- **Интеграция со сторонними сервисами:** встраивание платежных шлюзов (ЮKassa, Stripe), авторизация пользователей через социальные сети (Google, VK ID), интеграция интерактивных карт (Яндекс, Google Maps).
- **Клиент-серверная архитектура:** API служит связующим звеном, по которому мобильное приложение или веб-фронтенд запрашивают информацию из общей базы данных на сервере.
- **Доступ к динамическим данным:** автоматический сбор информации о погоде, курсах валют, биржевых котировках.

Билет №2

1. Использование командной строки Python Shell. Основные принципы работы с аргументами командной строки:

Python Shell (REPL — Read-Eval-Print Loop) — это интерактивная среда выполнения кода, запускаемая непосредственно из терминала командой `python` или `python3`. Среда позволяет выполнять инструкции построчно, сразу отображая результат. Она незаменима для быстрого тестирования гипотез, отладки небольших фрагментов логики или проведения математических расчетов. Выход из оболочки осуществляется функцией `exit()` или комбинацией клавиш `Ctrl+D`.

При запуске полноценных скриптов из консоли (например, `python script.py arg1 arg2`) параметры командной строки передаются в исполняемую программу. На базовом уровне доступ к ним можно получить с помощью встроенного списка `sys.argv` из модуля `sys`. В этом списке элемент `sys.argv[0]` всегда хранит имя запускаемого скрипта, а элементы с индексами 1 и далее содержат переданные строковые аргументы.

Для создания профессионального интерфейса командной строки с поддержкой флагов, типов данных, обязательных параметров и автоматической генерации справочной информации (хелпов) используется стандартная библиотека `argparse`.

2. Технология написания бота на языке Python:

Технология создания ботов базируется на взаимодействии с внешним программным интерфейсом платформы (например, Telegram Bot API) посредством HTTP-запросов. Существует два основных архитектурных метода получения новых сообщений и обновлений от серверов мессенджера:

1. **Long Polling:** скрипт бота работает в непрерывном цикле и регулярно посылает запросы на сервер платформы, спрашивая: «Появились ли новые сообщения?». Метод прост в отладке на локальном компьютере, но создает избыточную сетевую нагрузку.
2. **Webhooks:** бот разворачивается на сервере с публичным IP-адресом и SSL-сертификатом. При получении нового сообщения сервер платформы сам моментально отправляет POST-запрос на указанный URL-адрес бота. Это наиболее эффективный и промышленный метод.

В Python разработка ведется преимущественно с использованием специализированных библиотек. Наиболее популярной и современной является `aiogram` — полностью асинхронный фреймворк. Альтернатива — синхронная библиотека `pyTelegramBotAPI` (`telebot`). Логика программы строится на хэндлерах (обработчиках событий) — специальных декораторах, которые отслеживают входящие команды (например, `/start`), определенный текст или медиафайлы и вызывают подвязанные к ним функции.

| Билет №3

1. Типы данных языка Python:

Python относится к языкам с динамической, но строгой (сильной) типизацией. Динамическая означает, что тип переменной связывается с объектом, а не с именем, и может меняться в процессе работы. Строгая типизация запрещает неявные автоматические преобразования несовместимых типов (например, нельзя сложить строку с числом, не приведя их к общему типу).

Основные встроенные типы данных разделяют на несколько категорий:

- **Числовые:** целые числа произвольной точности (`int`), вещественные числа с плавающей точкой (`float`) и комплексные числа (`complex`).

- **Логический:** тип `bool`, принимающий только два константных значения: `True` (истина) и `False` (ложь).
- **Строковый:** тип `str` — неизменяемая последовательность символов в кодировке Unicode.
- **Коллекции / Структуры данных:**
 - `list` (список) — упорядоченная, изменяемая последовательность элементов;
 - `tuple` (кортеж) — упорядоченная, но строго неизменяемая последовательность;
 - `dict` (словарь) — изменяемая структура в формате пар «ключ: значение», построенная на хэш-таблицах;
 - `set` (множество) — неупорядоченная коллекция уникальных элементов.
- **Специальный тип:** `NoneType` (единственное значение — `None`), символизирующий отсутствие значения.

2. Инструментальные средства для разработки кроссплатформенных приложений на Python:

Кроссплатформенность в разработке ПО означает написание единой кодовой базы, которая без существенных изменений способна компилироваться и стабильно работать в операционных системах Windows, macOS, Linux, а также на мобильных платформах Android и iOS.

Основные инструменты разработки графического интерфейса (GUI) на Python:

- **Kivy:** бесплатная библиотека с открытым кодом, ориентированная на быструю разработку приложений с поддержкой инновационных пользовательских интерфейсов (включая multi-touch). Она не использует нативные элементы ОС, а отрисовывает собственный интерфейс с помощью графического движка поверх OpenGL ES, что обеспечивает идентичный вид на любой платформе.
- **KivyMD:** популярная библиотека-надстройка над Kivy. Она предоставляет разработчикам готовый набор высококачественных компонентов UI, полностью соответствующих гайдлайнам Google Material Design.
- **Flet:** современный инструмент, позволяющий создавать интерактивные приложения на Python, используя под капотом движок Flutter от Google.
- Для упаковки готового Python-кода в нативные установочные пакеты используются специальные утилиты: **Buildozer** (автоматизирует создание файлов `.apk` и `.aab` для Android) и **PyInstaller** (собирает автономные исполняемые файлы `.exe` / `.app` для настольных ПК).

| Билет №4

1. Работа с переменными в языке Python:

Переменная в Python концептуально отличается от классических компилируемых языков программирования (таких как C++ или Java). Переменная здесь — это не ячейка памяти фиксированного типа, а именованная ссылка (указатель) на конкретный объект, созданный в динамической памяти. Сам процесс объявления переменной совмещен с инициализацией — выделением памяти и связыванием имени с объектом посредством оператора присваивания (`=`).

Благодаря динамической типизации, одна и та же переменная на разных этапах выполнения алгоритма может ссылаться на объекты совершенно разных классов:

```
x = 10          # x указывает на объект int
x = "Hello"     # теперь x указывает на объект str
```

Правила именования переменных: имя может включать буквы латинского алфавита (как строчные, так и заглавные), цифры и символ нижнего подчеркивания `_`. Имя переменной категорически не может начинаться с цифры. Регистр символов имеет значение (`my_var` и `My_Var` — две разные переменные). Запрещено использовать зарезервированные ключевые слова синтаксиса языка (`if`, `while`, `class`, `import`). Официальный стандарт стиля PEP 8 рекомендует использовать для переменных строчные буквы с разделением слов подчеркиваниями (стиль `snake_case`).

2. Фреймворк Kivy, язык KV и виджеты, как основа пользовательского интерфейса:

Фреймворк Kivy спроектирован для создания гибких пользовательских интерфейсов. В основе его философии лежит строгое разделение логики работы приложения и декларативного описания его внешнего вида. Для реализации этой концепции был создан специализированный язык разметки — **KV Language (язык KV)**.

В файлах с расширением `.kv` интерфейс описывается в виде древовидной структуры. Это избавляет Python-код от нагромождения инструкций по позиционированию, цвету и размерам элементов. Язык KV поддерживает декларативное связывание данных: если свойство объекта в Python меняется, UI автоматически перерисовывается.

Виджеты (Widgets) — это неделимые графические элементы и контейнеры, из которых собирается интерфейс. Базовые виджеты UI включают: `Label` (вывод текста), `Button` (кнопка, генерирующая события при клике), `TextInput` (поле ввода текстовых данных пользователем), `Image` (вывод картинок) и `Slider` (ползунок). Компонуя виджеты внутри специальных макетов-контейнеров, разработчик формирует иерархию интерфейса.

| Билет №5

1. Встроенные функции языка Python:

Встроенные функции — это базовый набор процедур, которые встроены в ядро интерпретатора и доступны в любой части программы по умолчанию без необходимости явного импорта каких-либо дополнительных модулей. Их можно разделить по функциональному назначению:

- **Функции ввода и вывода информации:** `print(*objects)` — выводит переданные объекты в консоль; `input(prompt)` — приостанавливает выполнение алгоритма и ожидает ввода текстовой строки пользователем из консоли.
- **Функции приведения (конвертации) типов данных:** `int()`, `float()`, `str()`, `list()`, `dict()`, `set()`, `tuple()` — преобразуют переданный объект в соответствующий целевой тип, если это технически возможно.
- **Математические и агрегирующие функции:** `abs(x)` — модуль числа; `round(x)` — округление до ближайшего целого; `min()` и `max()` — поиск минимального и максимального элементов последовательности; `sum(iterable)` — вычисление суммы элементов.
- **Служебные функции исследования объектов:** `len(s)` — возвращает длину или количество элементов коллекции; `type(obj)` — возвращает класс (тип) объекта; `dir(obj)` — возвращает список всех доступных атрибутов и методов объекта.
- **Функции итерации:** `range(start, stop, step)` — генерирует неизменяемую арифметическую последовательность чисел.

2. Зарезервированные слова и выражения в языке KV:

В декларативной среде языка KV зарезервированы специальные ключевые слова-указатели. Они необходимы для динамической адресации объектов внутри иерархического дерева виджетов и связи разметки с логикой на Python:

- **self:** указывает на контекст текущего конкретного виджета, внутри описания которого пишется код. Пример: инструкция `text: self.state` внутри `Button` позволяет менять текст кнопки в зависимости от её состояния.
- **root:** ссылается на корневой виджет текущего блочного правила разметки. Корневым считается верхний родительский элемент в рамках данного KV-блока.
- **app:** указывает на глобальный запущенный экземпляр основного класса приложения (наследника `App` или `MDApp`). Это ключевое слово позволяет напрямую вызывать методы бизнес-логики, прописанные в главном Python-файле.
- **id:** уникальный текстовый идентификатор виджета. Он не отображается визуально, но позволяет другим элементам или коду Python мгновенно находить данный виджет в дереве и управлять его свойствами.

Билет №6

1. Основные операторы языка Python:

Операторы — это зарезервированные символы и конструкции языка, выполняющие операции над операндами. В Python операторы классифицируются по типам:

- **Арифметические операторы:** сложение (+), вычитание (-), умножение (*), деление с результатом с плавающей точкой (/), деление нацело с отбрасыванием дробной части (/ /), остаток от деления (%) и возведение в степень (**).
- **Операторы сравнения (возвращают bool):** строго равно (==), не равно (!=), строго больше (>), строго меньше (<), больше или равно (>=), меньше или равно (<=).
- **Логические операторы (для составных условий):** логическое И (and), логическое ИЛИ (or), логическое отрицание НЕ (not).
- **Операторы присваивания:** базовое (=) и комбинированные арифметические операторы (+=, -=, *=, /=). Инструкция `x += 5` эквивалентна `x = x + 5`.
- **Операторы членства и тождественности:** оператор `in` проверяет присутствие элемента внутри коллекции. Оператор `is` проверяет идентичность объектов (равенство их адресов в памяти `id()`).

2. Компоновка приложения из фрагментов методом соглашения имен (в Kivy):

В Kivy реализован элегантный механизм автоматического связывания управляющей логики на Python и визуального интерфейса на языке KV. Этот механизм называется «соглашением об именах» (Naming Convention) и работает на этапе вызова метода `run()` главного класса.

Алгоритм работы соглашения: интерпретатор берет имя главного класса вашего приложения (который наследуется от базового класса `App` или `MDApp`). Если имя класса оканчивается на слово `App`, фреймворк автоматически отбрасывает это слово. Оставшуюся часть названия переводит в нижний регистр (lowercase). После этого Kivy ищет в директории скрипта файл с полученным именем и расширением `.kv`. Если такой файл существует, он автоматически компилируется и внедряется в качестве корневой разметки окна.

Пример: если главный класс объявлен как `class WeatherTrackerApp(App):`, то Kivy отбросит суффикс `App`, получит строку `weathertracker` и автоматически загрузит файл интерфейса с точным именем `weathertracker.kv`. Это избавляет от необходимости прописывать ручную команду `Builder.load_file()`.

Билет №7

1. Методы работы с типами данных: строка, число:

Поскольку строки в Python являются неизменяемыми объектами (`immutable`), любой строковый метод не модифицирует исходную строку на месте, а создает и возвращает новый объект-строку. Основные методы типа `str`:

- **Управление регистром:** `.upper()` (все буквы заглавные), `.lower()` (строчные), `.capitalize()` (первая буква строки заглавная).
- **Поиск и замена:** `.find(sub)` (возвращает наименьший индекс вхождения подстроки или `-1`), `.replace(old, new)` (заменяет старые подстроки на новые), `.count(sub)` (подсчет числа вхождений).
- **Разделение и сборка:** `.split(sep)` — разрезает строку по разделителю в список; `sep.join(list)` — склеивает список строк в единый текст с использованием указанного разделителя.
- **Очистка:** `.strip()` — обрезает пробельные символы и переносы строк с краев текста.

Для работы с числами (`int`, `float`) чаще всего применяются математические операторы. Однако для сложных вычислений подключают стандартную библиотеку `math`, предоставляющую методы: `math.ceil(x)` (округление вверх), `math.floor(x)` (округление вниз), `math.sqrt(x)` (квадратный корень), `math.sin(x)` (тригонометрический синус).

2. Виджеты в Kivy:

Архитектура графического интерфейса Kivy разделяет все визуальные компоненты на две фундаментальные группы, каждая из которых наследуется от базового класса `Widget`:

1. **UX-виджеты (User Experience Widgets / Элементы управления):** это конечные функциональные элементы интерфейса, предназначенные для визуализации информации или непосредственного взаимодействия с пользователем. Они улавливают клики, касания и ввод. Сюда относятся текстовые поля `Label`, кнопки `Button`, интерактивные поля ввода `TextInput`, переключатели `CheckBox`, ползунки `Slider`, индикаторы загрузки `ProgressBar` и элементы вывода графики `Image`.
 2. **Макеты (Layouts / Контейнеры):** специализированные элементы, которые не имеют собственного визуального представления (они прозрачны), но выступают в роли родительских контейнеров. Их единственная задача — автоматическое управление пространством, размерами и координатами вложенных в них дочерних виджетов при изменении размеров экрана. Основные макеты: `BoxLayout` (линейное выравнивание), `GridLayout` (таблица), `FloatLayout` (абсолютная свобода позиционирования).
-

Билет №8

1. Использование операторов языка Python:

Использование операторов лежит в основе составления выражений. Интерпретатор строго следует правилам приоритета операторов: первыми вычисляются выражения в скобках, затем оператор возведения в степень (**), далее выполняются умножение и деление ($*$, $/$, $//$, $\%$), после них — сложение и вычитание ($+$, $-$), следом идут операторы сравнения и, наконец, логические операторы (`not`, `and`, `or`).

Благодаря концепции полиморфизма и механизму перегрузки операторов (магические методы), поведение одного и того же оператора кардинально меняется в зависимости от того, к объектам каких классов он применяется. Например:

```
print(5 + 5)      # Выведет 10 (математическое сложение int)
print("5" + "5")  # Выведет "55" (конкатенация объектов str)
print([1] * 3)    # Выведет [1, 1, 1] (дублирование списка list)
```

Такое поведение делает код лаконичным, но требует от программиста четкого понимания типов объектов, участвующих в выражении.

2. Задание размеров и положения виджетов в окне приложения Kivy:

По умолчанию Kivy использует адаптивную верстку, динамически вычисляя геометрию элементов на основе подсказок размеров (`size hints`) и подсказок положения (`pos hints`):

- `size_hint_x` и `size_hint_y`: принимают числовые значения от 0.0 до 1.0. Они указывают, какую относительную долю (процент) от доступной ширины и высоты родительского контейнера должен занять данный виджет. Например, `size_hint: (0.5, 0.2)` означает, что виджет займет 50% ширины и 20% высоты своего родителя.
- `pos_hint`: это управляющий словарь, привязывающий стороны или центр виджета к координатам родителя (например, `pos_hint: {'center_x': 0.5, 'center_y': 0.5}` идеально отцентрирует элемент).

Если макет требует задать жесткий фиксированный размер виджета в пикселях или аппаратно-независимых точках (`dp`), адаптивный механизм необходимо принудительно отключить. Для этого свойствам `size_hint_x` и `size_hint_y` присваивается значение `None`, после чего прописываются точные размеры через свойства `width`, `height` или кортеж `size`:

```
Button:
    size_hint: (None, None)
    width: "150dp"
    height: "50dp"
```

| Билет №9

1. Условные конструкции:

Условные конструкции управляют ветвлением логики выполнения программы на основе результатов проверки условий. Базовый синтаксис включает ключевые слова `if` (если), `elif` (иначе если — количество таких блоков не ограничено) и завершающий необязательный блок `else` (иначе — выполняется, если ни одно из вышестоящих условий не вернуло `True`).

```
if score >= 90:
    grade = "A"
```

```
elif score >= 75:
    grade = "B"
else:
    grade = "C"
```

Границы блоков кода в Python определяются строго величиной отступов (стандарт — 4 пробела). Для компактного присваивания значений используется тернарный условный оператор: значение_если_истина if условие else значение_если_ложь. Начиная с версии Python 3.10, в язык добавлен мощный инструмент структурного сопоставления шаблонов — конструкция `match / case`, являющаяся продвинутым аналогом оператора `switch-case` из других языков.

2. Задание виджетам цвета фона:

Подход к изменению фонового цвета в Kivy зависит от базовой природы виджета:

1. **Виджеты без собственного графического фона (например, Label):** такие элементы изначально прозрачны. Чтобы задать им цвет подложки, необходимо обратиться к инструкциям рисования графического контекста — холсту виджета. Код пишется внутри блока `canvas.before` (отрисовка до вывода основного контента). Там объявляется объект цвета `Color` в формате RGBA (значения каналов от 0 до 1) и геометрическая фигура `Rectangle`, размеры и координаты которой строго связываются с текущими размерами самого виджета:

```
Label:
    text: "Текст"
    canvas.before:
        Color:
            rgba: (0.1, 0.5, 0.9, 1) # Синий фон
        Rectangle:
            pos: self.pos
            size: self.size
```

2. **Виджеты с готовыми нативными текстурами (например, Button):** кнопка использует серое изображение-шаблон для отрисовки состояний. Свойство `background_color` работает как цветовой фильтр поверх этой текстуры, что искажает чистый цвет. Чтобы получить точный цвет, сначала сбрасывают дефолтную картинку пустой строкой: `background_normal: ''`, после чего задают цвет.

| Билет №10

1. Циклические алгоритмы:

Циклы предназначены для многократного повторения выполнения определенного блока инструкций. В Python реализованы две основные циклические конструкции:

- **Цикл `while`:** проверяет условие перед каждой итерацией. Тело цикла выполняется до тех пор, пока условие остается истинным. Если условие ложно изначально, цикл не выполнится ни разу. Важно изменять состояние переменных внутри тела, чтобы избежать бесконечного заикливания.
- **Цикл `for`:** это цикл обхода последовательностей (итератор). Он поочередно извлекает элементы из любого итерируемого объекта (списка, кортежа, строки, словаря) или числового диапазона, генерируемого функцией `range()`.

Управление циклами осуществляется специальными операторами: `break` — осуществляет мгновенный аварийный выход из цикла; `continue` — досрочно завершает текущую итерацию, пропуская оставшийся код тела цикла, и переходит к проверке условия следующего шага. Python поддерживает уникальную конструкцию `else` после блока цикла. Код в блоке `else` выполнится только тогда, когда цикл завершил свою работу штатно (без прерывания оператором `break`).

2. Обработка событий виджетов:

Графические интерфейсы работают на событийной архитектуре (Event-driven). Событие генерируется операционной системой или фреймворком при действиях пользователя: клике на кнопку, перемещении мыши, вводе символа или изменении размера окна приложения.

Способы обработки событий в Kivy:

- **Декларативный способ (в языке KV):** наиболее лаконичный вариант. В свойствах виджета прописывается атрибут-событие с префиксом `on_`, после которого через двоеточие указывается исполняемый код или вызов метода Python-класса:

```
Button:
    text: "Сохранить"
    on_release: app.save_data_method() # Срабатывает при отпускании кнопки
```

- **Императивный способ (в коде Python):** используется при динамическом создании элементов интерфейса в коде. Связывание события и функции-обработчика (callback) осуществляется вызовом специального метода `.bind()`:

```
new_button = Button(text="Жми")
new_button.bind(on_press=my_callback_function)
```

Функция-обработчик обязана принимать аргументом объект виджета, который сгенерировал данное событие (обычно именуется как `instance`).

| Билет №11

1. Структуры данных языка Python: списки, словари. Методы работы со списками, словарями:

Список (list) — это динамический упорядоченный изменяемый массив, способный хранить объекты любых типов в рамках одной коллекции. Элементы индексируются с нуля. Основные методы работы со списками:

- `.append(item)` — добавляет элемент в самый конец списка;
- `.insert(index, item)` — вставляет элемент в указанную позицию по индексу;
- `.remove(item)` — находит и удаляет первое вхождение указанного элемента;
- `.pop(index)` — удаляет и одновременно возвращает элемент по указанному индексу (по умолчанию удаляет последний);
- `.sort()` — сортирует элементы списка на месте по возрастанию.

Словарь (dict) — это изменяемая неупорядоченная (начиная с Python 3.7 — сохраняющая порядок добавления) структура данных, хранящая информацию в формате пар «ключ: значение». Построена на механизме хэш-таблиц, что гарантирует сверхбыстрый поиск значений за константное время

$O(1)$. Ключами могут выступать исключительно неизменяемые (хэшируемые) типы данных (числа, строки, кортежи). Методы словарей:

- `.get(key, default)` — безопасно возвращает значение по ключу. Если ключ отсутствует, не вызывает ошибку `KeyError`, а возвращает `None` или заданное дефолтное значение;
- `.keys()`, `.values()`, `.items()` — возвращают динамические представления ключей, значений или кортежей пар соответственно;
- `.update(other_dict)` — обновляет текущий словарь, добавляя пары из другого словаря.

2. Дерево виджетов – как основа пользовательского интерфейса:

Пользовательский интерфейс любого приложения Kivy представляет собой иерархическую структуру, организованную по принципу классического ориентированного графа — **дерева виджетов (Widget Tree)**. На самой вершине этой иерархии располагается единственный корневой виджет (Root widget). Данный виджет возвращается главной функцией `build()` или загружается автоматически из KV-файла при старте.

Корневой виджет обычно является макетом-контейнером верхнего уровня. В него вкладываются дочерние элементы — другие макеты или конечные UX-компоненты. Те, в свою очередь, могут содержать собственные вложенные поддеревья. Такая организация решает ключевые задачи фреймворка:

- **Каскадное распределение геометрии:** при изменении размеров экрана родительский узел пересчитывает доступную площадь и спускает новые размеры вниз своим потомкам.
- **Диспетчеризация событий (Event Bubbling):** сигналы касаний экрана (touch events) передаются строго по дереву сверху вниз, позволяя каждому слою проверить, попал ли клик в его геометрические границы.

| Билет №12

1. Объекты – кортеж, множества:

Кортеж (tuple) — это упорядоченная, но строго **неизменяемая** коллекция объектов произвольных типов. Синтаксически создается с помощью круглых скобок (например, `t = (1, "apple", True)`). Так как кортеж является иммутабельным, его невозможно модифицировать после создания — нельзя добавлять, удалять или заменять элементы. Это свойство обеспечивает две важные возможности: защиту критически важных данных от случайного изменения в ходе выполнения программы и экономию оперативной памяти (кортежи компактнее списков). Кроме того, благодаря хэшируемости, кортежи можно использовать в качестве ключей для словарей.

Множество (set) — это неупорядоченная изменяемая коллекция, хранящая исключительно **уникальные** элементы. Создается с помощью фигурных скобок (например, `s = {1, 2, 3}`). Попытка добавить дубликат элемента игнорируется. Множества построены на хэш-таблицах, благодаря чему проверка вхождения элемента через оператор `in` выполняется моментально. Множества поддерживают классические математические операции над множествами: объединение (`|`), пересечение (`&`), разность (`-`) и симметричную разность (`^`).

2. Виджеты для позиционирования элементов интерфейса в приложениях на Kivy:

Для управления взаимным расположением элементов интерфейса в Kivy используются специализированные виджеты-контейнеры, называемые макетами (Layouts). Они берут на себя обязанность рассчитывать координаты элементов. Основные виды макетов:

- **BoxLayout**: базовый макет линейного позиционирования. Он выстраивает добавленные в него виджеты строго один за другим в один ряд. Направление ряда задается свойством `orientation` и может быть горизонтальным ('horizontal', по умолчанию) или вертикальным ('vertical').
- **GridLayout**: табличный макет. Он распределяет виджеты по регулярной сетке. Разработчик обязан жестко зафиксировать либо количество столбцов с помощью свойства `cols`, либо количество строк через свойство `rows`, после чего макет сам заполнит ячейки по порядку.
- **AnchorLayout**: макет привязки. Он позволяет «прижать» дочерний элемент к какому-либо конкретному краю экрана или выровнять его строго по центру. Настраивается параметрами `anchor_x` (left, center, right) и `anchor_y` (top, center, bottom).
- **FloatLayout**: макет абсолютного позиционирования. Предоставляет полную свободу действий. Виджеты размещаются в любых координатах окна, а их позиция и размеры чаще всего задаются через процентные подсказки `pos_hint` и `size_hint`.

Билет №13

1. Понятие функции. Конструкции и методы создания функций в языке Python:

Функция — это изолированный, именованный фрагмент программного кода, предназначенный для выполнения строго определенной подзадачи. Функции являются фундаментальным инструментом структурного программирования, позволяя разбивать громоздкий алгоритм на декомпозированные блоки. Использование функций реализует важнейший принцип разработки ПО — DRY (Don't Repeat Yourself), исключая дублирование идентичного кода.

Создание функции: объявляется с помощью ключевого слова `def` (define), за которым следует уникальное имя функции, круглые скобки для перечисления входных параметров и двоеточие. Тело функции записывается на следующем блочном уровне с обязательным отступом.

```
def calculate_area(width, height):  
    area = width * height  
    return area
```

Для возврата вычисленного результата обратно в вызывающую программу используется оператор `return`. Инструкция `return` мгновенно завершает выполнение функции. Если оператор явным образом не прописан в теле функции или после него нет значения, функция по умолчанию вернет специальный объект `None`.

2. Виджет Carousel для пролистывания слайдов:

Виджет `Carousel` (Карусель) — это специализированный высокоуровневый макет-контейнер в Kivy, предназначенный для организации экранов со сменяемыми слайдами. Он позволяет пользователю перелистывать элементы интерфейса свайп-движением (пальцем на сенсорном экране или мышью на десктопе).

Направление пролистывания гибко регулируется встроенным свойством `direction`, которое принимает значения 'right', 'left', 'top' или 'bottom'. В качестве слайдов внутрь Карусели

можно добавлять абсолютно любые виджеты, изображения или сложные вложенные макеты с помощью стандартного метода `.add_widget()`. Основные управляющие свойства:

- **loop**: логическое свойство (True/False). При включении переводит карусель в циклический режим (после последнего слайда пользователь снова видит первый);
- **index**: целочисленное свойство, отражающее порядковый номер текущего активного слайда;
- Программное переключение слайдов (например, по нажатию на отдельные стрелочки-кнопки) реализуется встроенными методами объекта `.load_next()` (загрузить следующий слайд) и `.load_previous()` (загрузить предыдущий).

Билет №14

1. Передача аргументов в параметры функций. Лямбда функции в языке Python:

При вызове функций передача значений (аргументов) в переменные функции (параметры) может осуществляться различными путями:

- **Позиционные аргументы**: передаются в функцию строго в том порядке, в каком параметры были объявлены в заголовке функции (например, при `func(1, 2)` единица уйдет в первый параметр, двойка — во второй).
- **Именованные (ключевые) аргументы**: передаются с явным указанием имени параметра через знак равенства (например, `func(height=100, width=50)`). При таком подходе их последовательность при вызове не имеет значения.
- **Параметры со значениями по умолчанию**: задаются на этапе объявления функции (например, `def connect(host="localhost")`). Если при вызове аргумент опущен, подставится дефолтное значение.
- **Гибкие параметры**: конструкция `*args` собирает избыточные позиционные аргументы в кортеж, а `**kwargs` собирает лишние именованные аргументы в словарь.

Лямбда-функции (анонимные): это короткие вспомогательные функции, создаваемые в однострочном формате без присвоения им постоянного имени. Объявляются ключевым словом `lambda`. Синтаксис: `lambda параметры: выражение`. Они не содержат оператора `return`, но автоматически возвращают результат вычисления выражения. Пример: `square = lambda x: x ** 2`. Применяются в местах, где логика проста и требуется разово внутри другого выражения.

2. Задание цвета фона виджету – контейнеру:

Контейнеры макетов (такие как `BoxLayout`, `GridLayout` и др.) спроектированы в Kivy исключительно как логические инструменты распределения пространства. Они изначально полностью прозрачны и не имеют собственных визуальных свойств вроде `"background_color"`. Задание им фонового цвета требует ручного добавления графических примитивов на их фоновый холст — `canvas.before`.

Важнейшим правилом верстки в Kivy является обеспечение реактивности графики. Если приложение будет запущено на устройстве с другим разрешением или пользователь перевернет экран смартфона, контейнер изменит свои размеры, а нарисованный фон должен автоматически подстроиться. Для этого свойства позиции и размера прямоугольника жестко связываются с динамическими свойствами самого контейнера через зарезервированное слово `self`:

```
BoxLayout:
    orientation: 'vertical'
```

```
canvas.before:
    Color:
        rgba: (0.15, 0.15, 0.15, 1) # Темно-серый цвет фона
    Rectangle:
        pos: self.pos # Фон всегда смещается вслед за контейнером
        size: self.size # Фон всегда равен точным размерам контейнера
```

Билет №15

1. Анонимные функции и их применение внутри функций высшего порядка:

Функции высшего порядка (ФВП) — это функции, которые принимают другие функции в качестве входных аргументов, либо возвращают функцию в качестве результата своей работы. В функциональной парадигме Python анонимные лямбда-функции выступают идеальным дополнением для ФВП, позволяя писать компактный, декларативный код без загромождения глобальной области видимости мелкими именованными функциями `def`.

Классические примеры применения лямбда-функций внутри ФВП в Python:

- `map(function, iterable)`: последовательно применяет функцию к каждому элементу коллекции.

```
squares = list(map(lambda x: x**2, [1, 2, 3, 4])) # Результат: [1, 4, 9, 16]
```

- `filter(function, iterable)`: фильтрует элементы коллекции, оставляя только те, для которых функция вернула `True`.

```
evens = list(filter(lambda x: x % 2 == 0, [1, 2, 3, 4])) # Результат: [2, 4]
```

- `sorted(iterable, key)`: выполняет сортировку коллекции по кастомному критерию, генерируемому лямбдой.

```
points = [(1, 5), (3, 2), (2, 9)]
sorted_by_y = sorted(points, key=lambda p: p[1]) # Сортировка по второй координате
```

2. Индексация элементов в дереве виджетов:

Каждый родительский виджет или макет-контейнер в Kivy автоматически отслеживает список всех добавленных в него дочерних элементов. Этот список хранится во встроенном атрибуте-коллекции под именем `children`, который представляет собой обычный питоновский список.

Особенность индексации: элементы внутри списка `children` упорядочены в хронологическом порядке, противоположном порядку их добавления. Самый первый виджет, который был добавлен в макет (через код KV или методом `add_widget()`), оказывается в самом конце списка (имеет наибольший индекс). Соответственно, элемент, добавленный самым последним, занимает позицию с индексом `[0]`.

Знание этого правила необходимо для динамического управления интерфейсом из Python-кода. Например, если внутри макета `my_box` лежат две текстовые метки, и нам нужно изменить текст верхней (которая была добавлена последней), мы пишем: `self.ids.my_box.children[0].text = "Новый текст"`.

Билет №16 / №17

1. Работа синхронных функций. Создание асинхронной функции:

Синхронное выполнение: это классическая последовательная модель. Когда вызывается синхронная функция, выполнение основного потока программы полностью приостанавливается (блокируется). Интерпретатор послушно ждет, пока функция выполнит все свои инструкции и вернет управление. Если внутри графического интерфейса (GUI) вызвать синхронную функцию, выполняющую длительную операцию (скачивание файла большого размера, тяжелый запрос к базе данных), приложение "зависнет": оно перестанет реагировать на клики и перерисовывать экран, так как графический поток заблокирован.

Асинхронное выполнение: позволяет эффективно обрабатывать операции ввода-вывода (I/O-bound процессы) без блокировки основного потока. В Python это реализуется встроенной библиотекой `asyncio` и концепцией цикла событий (event loop).

Асинхронная функция создается добавлением ключевого слова `async def` перед её объявлением. Внутри такой функции при вызове длительных блокирующих операций применяется ключевое слово `await`. Инструкция `await` временно приостанавливает выполнение данной конкретной функции, отдавая управление обратно в event loop, который продолжает прорисовывать интерфейс или выполнять другие задачи, пока ожидается ответ от сети.

```
import asyncio

async def fetch_api_data():
    print("Старт запроса...")
    await asyncio.sleep(3) # Асинхронное ожидание, поток не заблокирован
    print("Данные успешно получены!")
    return {"status": 200}
```

2. Классы `Screen` и `ScreenManager` для создания многоэкранных приложений:

Современные мобильные и десктопные приложения редко состоят из одного статичного окна. Обычно интерфейс разделен на логические экраны (например: Экран авторизации, Главная страница, Окно настроек, Корзина). В Kivy за построение такой архитектуры отвечают два взаимосвязанных класса из модуля `kivy.uix.screenmanager`:

- **Класс `Screen`:** представляет собой изолированный визуальный слой (отдельный экран). Он наследуется от обычного виджета, поэтому внутри него можно создавать любую древовидную структуру макетов и кнопок. Каждый создаваемый экран в обязательном порядке должен обладать уникальным строковым свойством `name`, которое служит его главным адресом.
- **Класс `ScreenManager`:** это управляющий контейнер-менеджер верхнего уровня. Он регистрирует в себе коллекцию созданных экранов `Screen`. Переключение между экранами осуществляется простым изменением свойства `current` менеджера экранов (например, строка `screen_manager.current = 'settings'` мгновенно скроет текущий экран и отобразит окно настроек). Класс также поддерживает управление встроенными анимациями переходов (`transition`) — например, плавное скольжение экрана (`SlideTransition`) или растворение (`FadeTransition`).

Билет №18

1. Возможности подключаемых библиотек языка Python:

Ключевым преимуществом Python, обеспечившим ему глобальную популярность, является философия «батарейки в комплекте» (встроенная мощная библиотека) в сочетании с колоссальной экосистемой сторонних пакетов, разрабатываемых мировым сообществом. Подключаемый код расширяет возможности языка до любых прикладных задач.

Все библиотеки разделяют на две базовые группы:

1. **Стандартная библиотека (встроенные модули):** поставляется вместе с интерпретатором. Модуль `os` предоставляет инструменты взаимодействия с файловой системой ОС; `sys` управляет параметрами среды интерпретатора; `math` реализует математические функции; `json` парсит одноименный текстовый формат данных; `datetime` работает с временными метками.
2. **Сторонние библиотеки (внешние пакеты):** публикуются в глобальном репозитории PyPI (Python Package Index) и устанавливаются на компьютер разработчика одной консольной командой через менеджер пакетов `pip install имя_библиотеки`. Примеры: `requests` (сетевые HTTP-запросы), `pandas` (аналитика данных), `kivy/PyQt` (графические UI). Подключение кода в тело скрипта выполняется директивой `import`.

2. Класс Window для формирования окна приложения:

Класс `Window`, импортируемый из базового ядра фреймворка (`kivy.core.window`), является программной абстракцией над реальным физическим окном операционной системы, в котором разворачивается и отрисовывается приложение.

Этот класс предоставляет низкоуровневые инструменты для динамического управления параметрами отображения и перехвата системных сигналов:

- **Управление геометрией и внешним видом:** свойство `Window.size` позволяет жестко задать стартовую ширину и высоту окна на ПК в пикселях (например, `Window.size = (360, 640)` часто применяется для симуляции экрана смартфона при отладке мобильного интерфейса); свойство `Window.clearcolor` определяет базовый цвет очистки экрана (фон всего приложения).
- **Перехват клавиатурного ввода:** метод `Window.bind(on_key_down=...)` позволяет зарегистрировать callback-функцию, которая будет перехватывать нажатия любых физических клавиш на клавиатуре компьютера (это необходимо, например, для закрытия всплывающих окон по кнопке `Escape`).
- **Мобильная интеграция:** класс управляет поведением и мягким скрыванием/появлением встроенной экранной клавиатуры на устройствах под управлением Android и iOS.

Билет №19 / №20

1. Модули и основные функции библиотек `time` и `random`:

Оба модуля входят в состав стандартной библиотеки Python и обеспечивают базовую функциональность алгоритмов:

Модуль `time` (управление временем):

- `time.time()` — возвращает текущую системную временную метку в формате `Float`, выраженную в количестве секунд, прошедших с полуночи 1 января 1970 года (эпоха Unix). Широко применяется для замера точной скорости работы алгоритмов.
- `time.sleep(seconds)` — принудительно останавливает выполнение текущего потока программы на указанное число секунд (блокирующая функция).

Модуль **random** (генерация псевдослучайных величин):

- `random.random()` — генерирует случайное вещественное число (float) в полуоткрытом диапазоне от 0.0 до 1.0;
- `random.randint(a, b)` — возвращает случайное целое число из закрытого отрезка $[a, b]$ включительно;
- `random.choice(sequence)` — извлекает один случайный элемент из переданной непустой последовательности (например, случайное слово из списка);
- `random.shuffle(list)` — случайным образом перемешивает элементы изменяемого списка непосредственно на месте его хранения.

2. Структура приложений на KivyMD:

Архитектура графического приложения на базе KivyMD унаследована от Kivy, однако адаптирована под жесткие визуальные стандарты дизайн-системы Google Material Design. Структура типичного проекта включает следующие особенности:

- **Главный класс приложения:** вместо базового класса Kivy наследование производится от класса `kivymd.app.MDApp`. Именно он инициализирует контекст Material-дизайна.
- **Менеджер тем (`theme_cls`):** внутри объекта класса `MDApp` создается управляющий атрибут `theme_cls`. С его помощью можно одной строчкой кода менять визуальную гамму всего интерфейса. Свойство `theme_cls.theme_style` принимает значения "Light" или "Dark" (светлая/темная тема), а `primary_palette` задает стержневой цвет бренда приложения (например, "Blue", "Teal", "Red").
- **Специализированные виджеты:** все стандартные компоненты UI заменяются на продвинутые аналоги из пакета KivyMD, которые имеют префикс MD. Вместо `Button` используется `MDRaisedButton` (кнопка с тенью) или `MDIconButton` (кнопка-иконка), вместо `TextInput` — `MDTextField` (поле с красивой анимацией фокуса и валидацией текста). Корневыми слоями чаще всего назначаются контейнеры `MDScreen` и `MDScreenManager`.

| Билет №21

1. Открытие и сохранение файлов разных форматов с использованием библиотек языка Python:

Работа с внешними файлами в Python базируется на концепции потоков ввода-вывода и встроенной функции `open(file, mode, encoding)`. Промышленным стандартом является использование оператора контекстного менеджера `with`. Он гарантирует автоматическое и надежное закрытие файла после завершения операций, даже если внутри блока произойдет критический сбой алгоритма.

Текстовые файлы (.txt): открываются в режиме 'r' (read) для чтения или 'w' (write) для перезаписи контента. При работе с русскоязычными текстами обязательно прописывается кодировка `encoding='utf-8'`:

```
with open("data.txt", "w", encoding="utf-8") as file:
    file.write("Тестовая строка")
```

Формат JSON (.json): используется для сохранения структурированных данных (словарей и списков). Подключается стандартный модуль `json`. Метод `json.dump(obj, file)` сериализует питоновский объект в файл, а функция `json.load(file)` производит обратную десериализацию текстового JSON-файла обратно в структуру данных Python.

Табличные форматы (.csv): обрабатываются встроенным модулем `CSV` или мощной сторонней библиотекой `pandas`, методы которой `pd.read_csv()` и `df.to_csv()` позволяют сохранять и читать огромные таблицы в одну команду.

2. Структура многоэкранных приложений на основе менеджера экранов (ScreenManager):

Применение менеджера экранов формирует чистую архитектуру приложения, разделяя экраны на независимые модули. В кодовой структуре это реализуется декларативным объявлением дерева в KV-файле:

```
ScreenManager:
    LoginScreen:
    MainScreen:

<LoginScreen>:
    name: 'login_screen'
    BoxLayout:
        Button:
            text: "Войти"
            on_release: root.manager.current = 'main_screen'

<MainScreen>:
    name: 'main_screen'
    Label:
        text: "Добро пожаловать в систему!"
```

В этой структуре `ScreenManager` выступает в роли корневого виджета. Внутри него перечисляются классы создаваемых экранов. Фреймворк автоматически делает активным тот экран, который прописан первым по порядку.

Каждый дочерний объект `Screen` автоматически получает скрытую внутреннюю ссылку на своего родительского менеджера через свойство `root.manager` (или `self.manager`). Благодаря этому изнутри любого экрана (например, по нажатию на кнопку "Выйти" или "Войти") можно вызвать команду смены активного окна, передав строковое имя целевого экрана в параметр `current`.

Билет №22

1. Понятие объекта. Создание методов объекта и их полей:

В рамках Объектно-Ориентированного Программирования (ООП) **Объект** — это конкретный экземпляр класса, олицетворяющая сущность виртуального мира, которая обладает определенным индивидуальным состоянием (хранит данные) и поведением (способна выполнять алгоритмы). В Python концепция ООП возведена в абсолют: абсолютно любая сущность (будь то число, строка, список или функция) является полноценным объектом.

Шаблон-чертежом для создания объектов выступает `class`. Описание внутренней структуры объекта включает две составляющие:

- **Поля объекта (Атрибуты):** это переменные, жестко связанные с конкретным экземпляром и описывающие его характеристики. Создание полей выполняется внутри специального магического метода-конструктора `__init__`. Чтобы привязать переменную к объекту, ее имя обязательно прописывается через префикс `self`.

- **Методы объекта:** это функции, объявленные внутри тела класса. Они предназначены для обработки данных объекта или изменения его полей. Ключевым правилом синтаксиса Python является то, что любой метод экземпляра в обязательном порядке должен принимать в качестве первого входного аргумента ссылку на сам этот объект (по соглашению этот аргумент всегда именуется словом `self`).

```
class Smartphone:
    def __init__(self, model):
        self.model = model # Создание поля объекта

    def turn_on(self):      # Создание метода объекта
        print(f"Смартфон {self.model} включен.")
```

2. Стили KivyMD для задания цвета элементам интерфейса:

Дизайн-система KivyMD полностью отказывается от хаотичного ручного прописывания RGB-координат цвета для каждого отдельного виджета, так как это нарушает консистентность интерфейса. Цветовое оформление элементов базируется на системных стилях Material Design.

Для управления цветом применяются специализированные свойства компонентов (например, `md_bg_color` для заливки фона, `text_color` для шрифтов). В качестве значений эти свойства принимают динамические ссылки на палитру приложения:

- `app.theme_cls.primary_color` — главный бренд-цвет приложения, используемый для ключевых элементов, шапки (AppBar) и основных кнопок.
- `app.theme_cls.accent_color` — контрастный акцентный цвет для выделения важных интерактивных элементов (чекбоксы, активные ползунки, переключатели).
- Строковые константы Material Design: цвет можно задать именем стандартной палитры Google. Например, инструкция `md_bg_color: "Red"` или `md_bg_color: "Teal"` окрасит элемент в строго выверенный оттенок, гармонирующий с общими гайдлайнами дизайна.

Билет №23

1. Свойства ООП: понятие инкапсуляции, наследования, полиморфизма:

Три фундаментальных принципа (столпа) ООП определяют архитектуру программного обеспечения:

1. **Инкапсуляция:** механизм объединения данных (полей) и методов их обработки внутри одного класса, совмещенный с сокрытием внутренней реализации и защитой критических данных от прямого несанкционированного изменения из внешнего кода. В Python сокрытие носит рекомендательный характер и реализуется префиксами: одиночное подчеркивание (`_protected`) маркирует элемент, который не стоит трогать вне класса; двойное подчеркивание (`__private`) включает механизм name mangling, аппаратно затрудняя прямой доступ к полю. Контролируемый доступ к полям реализуют через геттеры и сеттеры (декоратор `@property`).
2. **Наследование:** возможность создания нового класса (дочернего / подкласса) на основе уже существующего (родительского / суперкласса). дочерний класс автоматически перенимает все поля и методы родителя, а также получает возможность добавлять собственные или переопределять существующие. Синтаксис: `class Child(Parent):`. Сокращает дублирование кода.
3. **Полиморфизм:** концепция, позволяющая использовать объекты разных классов с одинаковым интерфейсом (одинаковыми именами методов) без необходимости знать точный тип объекта на этапе

написания кода. В Python полиморфизм выражен принципом "утиной типизации" (Duck Typing): если объект ведет себя как утка (имеет нужный метод), интерпретатор выполнит код, невзирая на его официальное древо наследования.

2. Темы KivyMD для настройки цветовых оттенков:

Централизованное управление визуальной атмосферой в KivyMD возложено на объект-менеджер `theme_cls`, интегрированный в глобальный класс приложения `MdApp`. Изменение глобальных свойств этого менеджера мгновенно каскадно перерисовывает все виджеты на всех экранах приложения:

- `theme_cls.theme_style`: управляет общим световым тоном интерфейса. Принимает два фиксированных значения: "Light" (светлая тема — белый/светло-серый фон окон, темный текст) и "Dark" (темная тема — глубокий темно-серый фон, белый текст для снижения нагрузки на глаза в темноте). Переключение темы реализуется на лету одной командой.
- `theme_cls.primary_palette`: устанавливает базовую цветовую схему для ключевых компонентов. Разработчик выбирает одно из стандартных имен палитр Material Design (например, "Blue", "Green", "Purple", "Amber").
- `theme_cls.accent_palette`: определяет второстепенный цветовой оттенок, используемый для тонких графических акцентов интерфейса.

| Билет №24

1. Работа магических методов:

Магические методы (иначе именуемые как ****dunder-методы****, от англ. **double underscore** — двойное подчеркивание) — это специальные предопределенные методы внутри классов Python, имена которых строго начинаются и заканчиваются двумя символами нижнего подчеркивания (например, `__init__`). Их ключевая особенность заключается в том, что программист практически никогда не вызывает их явным образом вручную. Они вызываются встроенными механизмами самого интерпретатора Python автоматически при наступлении определенных системных триггеров.

Примеры работы фундаментальных магических методов:

- `__init__(self, ...)` — метод-конструктор. Он автоматически триггерится в памяти сразу после того, как был порожден новый экземпляр класса. Используется для первичной инициализации полей объекта начальными значениями.
- `__str__(self)` — автоматически вызывается, когда объект пытаются преобразовать к строковому типу с помощью функции `str(obj)` или выводят его на экран через стандартный `print(obj)`. Этот метод обязан возвращать лаконичный, понятный человеку текст, описывающий объект.
- `__repr__(self)` — генерирует официальное строковое представление объекта, которое в идеале должно выглядеть как валидный Python-код для воссоздания этого объекта. Используется разработчиками при логировании и отладке.

2. Компоненты KivyMD для позиционирования элементов интерфейса:

KivyMD расширяет базовый набор макетов Kivy, предоставляя разработчику специализированные контейнеры. Они адаптированы под Material Design и поддерживают нативные свойства стилизации (тени, скругления, управление цветом):

- `MdBoxLayout` и `MdGridLayout`: расширенные аналоги базовых макетов Kivy. Они сохраняют прежнюю логику линейного и сеточного размещения, но добавляют свойства быстрого управления

фоном (`md_bg_color`), внутренними отступами (`padding`) и межэлементными интервалами (`spacing`) прямо в Material-стиле.

- **MDFloatLayout**: свободный слой, используемый для наложения элементов друг на друга или создания сложных кастомных анимаций перемещения виджетов.
- **MDCard**: компонент графической карточки. Являясь полноценным контейнером, карточка позволяет группировать внутри себя другие виджеты (текст, картинки, кнопки). Она обладает встроенной поддержкой важнейшего свойства Material Design — **elevation** (физическая глубина/высота подъема элемента, за счет которой под карточкой отрисовывается реалистичная мягкая тень).

Билет №25

1. Переопределение действий магических методов:

Переопределение dunder-методов (также называемое ****перегрузкой операторов****) — это мощный механизм ООП в Python. Он позволяет научить объекты пользовательских классов стандартному поведению встроенных типов данных. Переопределяя магические методы, разработчик программирует реакцию своего объекта на базовые математические символы или встроенные функции языка.

Основные группы методов для переопределения:

- **Математические операторы**: метод `__add__(self, other)` перегружает оператор сложения (+); `__sub__` — вычитание (-); `__mul__` — умножение (*). Например, для класса "Вектор" или "Денежная сумма" можно переопределить метод `__add__`, чтобы складывать объекты напрямую через знак плюс: `vector3 = vector1 + vector2`.
- **Операторы сравнения**: метод `__eq__(self, other)` переопределяет логическую проверку на равенство (оператор ==); `__lt__` задает логику работы знака "меньше" (<); `__gt__` — знака "больше" (>). Это позволяет сравнивать объекты между собой или автоматически сортировать списки с кастомными объектами встроенной функцией `sorted()`.
- **Дополнительные**: `__len__(self)` позволяет объекту возвращать свою внутреннюю длину при вызове функции `len(obj)`.

2. Компоненты пользовательского интерфейса KivyMD:

Библиотека KivyMD предоставляет обширный инструментарий готовых, высокотехнологичных компонентов пользовательского интерфейса, спроектированных по спецификациям Material Design. Основные кирпичики интерфейса:

- **MDLabel**: текстовый компонент. Обладает продвинутым управлением шрифтами через свойство `font_style`, поддерживающим стандартные стили иерархии (Headline, Subtitle, Body1, Button).
- **Семейство кнопок**: **MDRaisedButton** (классическая кнопка с плотной заливкой цветом бренда и тенью), **MDFlatButton** (прозрачная текстовая кнопка без границ для диалоговых окон), **MDIconButton** (компактная круглая кнопка, содержащая только графическую иконку).
- **MDTextField**: интерактивное текстовое поле ввода. Поддерживает анимацию смещения подсказки (`hint_text`) вверх при фокусе, встроенный счетчик символов и автоматическую подсветку поля красным цветом при возникновении ошибок валидации.
- **MDTopAppBar**: важнейший структурный компонент — верхняя панель приложения. Хранит заголовок текущего экрана, кнопку вызова бокового меню (гамбургер) и контекстные кнопки действий.

Билет №26

1. Диаграммы деятельности. Рекомендации по построению диаграмм деятельности:

Диаграмма деятельности (Activity Diagram) в графическом стандарте UML представляет собой специализированный инструмент визуализации, отображающий процедурный поток управления и последовательность шагов бизнес-процесса или сложного логического алгоритма внутри программной системы. По сути, она является развитым объектным аналогом классической блок-схемы.

Основные графические элементы диаграммы: начальный узел (сплошной черный кружок), действие/активность (прямоугольник со скругленными углами), ветвление/принятие решений (ромб с исходящими альтернативными стрелками-условиями), параллельные потоки (жирные горизонтальные или вертикальные линии синхронизации) и финальный узел (черный кружок в рамке).

Рекомендации по грамотному проектированию диаграмм деятельности:

1. Диаграмма должна обладать строгой топологией — иметь ровно одну стартовую точку и логически завершаться в понятных финальных узлах.
2. Для разграничения зон ответственности за выполнение конкретных шагов используйте «плавательные дорожки» (Swimlanes). Каждая дорожка подписывается именем актера (пользователя) или компонента системы, выполняющего данное действие.
3. Стрелки переходов должны быть направлены преимущественно сверху вниз или слева направо для естественного восприятия хронологии процесса.
4. Избегайте чрезмерной детализации. Если процесс перегружен, выполняйте декомпозицию, вынося сложные подпроцессы на отдельные дочерние диаграммы.

2. Гибкие методологии проектирования:

Гибкие методологии разработки ПО (Agile) представляют собой современную философию и концептуальный каркас ведения инженерных проектов, альтернативный классической тяжеловесной «каскадной» модели (Waterfall). Потребность в Agile возникла из-за высокой динамичности ИТ-сферы, где требования заказчика к итоговому продукту часто и радикально меняются в процессе разработки.

Философия Agile была официально закреплена в 2001 году в «Манифесте гибкой разработки ПО», постулирующем четыре ключевые ценности:

- **Люди и их живое взаимодействие** ставятся выше жестко зафиксированных процессов и используемых инструментальных средств.
- **Работающий программный продукт** имеет наивысший приоритет и важнее составления исчерпывающей, детальной документации.
- **Постоянное сотрудничество с заказчиком** в ходе работы важнее строгого согласования и отстаивания первоначальных условий контракта.
- **Готовность к оперативным изменениям** и гибкой смене курса важнее следования первоначально утвержденному долгосрочному плану.

Agile не является конкретным сводом инструкций — это образ мышления. Практическая реализация гибкого подхода осуществляется конкретными методологиями и фреймворками, наиболее известными из которых являются Scrum и Kanban.

Билет №27

1. Диаграммы последовательности. Рекомендации по построению диаграмм последовательности:

Диаграмма последовательности (Sequence Diagram) в языке UML — это диаграмма взаимодействия, сфокусированная на временной шкале и отображающая детальную хронологию обмена информационными сообщениями между различными объектами, компонентами системы или пользователями (актерами) для успешной реализации конкретного прецедента использования (Use Case).

Графика диаграммы включает: объекты (прямоугольники в верхней части), вертикальные пунктирные линии жизни (Lifelines), уходящие вниз и символизирующие ход времени, фокусы активности (узкие вертикальные блоки на линиях жизни, показывающие время, когда объект выполняет работу) и горизонтальные стрелки сообщений (сплошная стрелка — синхронный вызов метода, пунктирная стрелка — возврат результата ответа).

Рекомендации по построению диаграмм последовательности:

1. Располагайте участников взаимодействия слева направо по мере их вовлечения в алгоритм. Как правило, крайним левым элементом выступает актер-пользователь, инициирующий процесс, а правее располагаются UI-слой, контроллеры, бизнес-логика и база данных.
2. Соблюдайте строгую хронологию: время движется исключительно сверху вниз. Пересечение стрелок сообщений по вертикали недопустимо.
3. Не документируйте очевидные операции. Опускайте тривиальные вызовы вроде геттеров/сеттеров, концентрируясь на ключевых архитектурных шагах.
4. Используйте блоки комбинированных фрагментов для логики: рамка `alt` визуализирует условное разветвление (if/else), рамка `loop` — циклы.

2. Agile методология:

Развивая концепцию гибкой разработки, важно зафиксировать, как Agile методология функционирует на практике. Весь проект разработки ПО разбивается на последовательность небольших временных отрезков — **итераций** (длительностью, как правило, от 1 до 4 недель).

В начале каждой итерации команда совместно с заказчиком определяет пул наиболее приоритетных задач из общего списка требований. В течение итерации эти задачи кодируются, тестируются и интегрируются. Главным правилом Agile является то, что в самый последний день итерации команда обязана продемонстрировать и передать заказчику ****инкремент продукта**** — полностью работоспособную версию программного обеспечения, содержащую новые функции.

Такой подход минимизирует финансовые и временные риски: заказчик видит продукт в динамике, может тестировать его на реальных пользователях и своевременно корректировать требования для следующих итераций. Это полностью исключает ситуацию, когда команда тратит год на разработку по ТЗ, а на выходе получается продукт, потерявший актуальность для рынка.

Билет №28

1. Требования к интерфейсу пользователя. Принципы создания графического пользовательского интерфейса (GUI):

Интерфейс пользователя (UI) — это техническая среда, посредством которой человек взаимодействует с программным комплексом. Качественный интерфейс должен удовлетворять жестким критериям оценки: ****интуитивность**** (пользователь понимает логику без чтения инструкций), ****отзывчивость**** (минимальное время отклика системы на клики), ****доступность****, ****лаконичность**** (отсутствие визуального шума) и ****согласованность****.

При проектировании GUI разработчики опираются на фундаментальные инженерные принципы:

- **Принцип ясности и видимости:** текущее состояние системы должно быть прозрачным для человека. Если идет загрузка данных, обязательно отображается ProgressBar или спиннер, информирующий о процессе.
- **Принцип контроля и свободы:** пользователь должен чувствовать себя хозяином положения. Каждое действие должно быть обратимым — обязательное наличие функций отмены (Undo) и кнопок "Назад" / "Выход".
- **Принцип консистентности (единообразия):** элементы управления, имеющие идентичную функциональность, обязаны выглядеть одинаково на всех экранах приложения. Шрифты, иконки, логика расположения меню и цветовая палитра подчиняются единому стандарту (дизайн-системе).
- **Принцип предотвращения ошибок:** интерфейс должен проектироваться так, чтобы минимизировать саму возможность неверного действия пользователя (например, кнопка "Отправить форму" должна оставаться заблокированной и серой до тех пор, пока все обязательные поля ввода не пройдут корректную валидацию).

2. Scrum методология:

Scrum — это наиболее структурированный, популярный фреймворк, реализующий принципы философии Agile. Он организует работу кросс-функциональной команды разработчиков в виде строго регламентированных циклов, называемых ****Спринтами**** (фиксированная длительность от 1 до 4 недель, чаще всего — 2 недели).

В Scrum жестко распределены роли между участниками процесса:

- **Product Owner (Владелец продукта):** ключевой человек, представляющий интересы заказчика, клиентов и бизнеса. Он единолично формирует требования к системе, ранжирует их по степени важности и ведет Бэклог продукта (Product Backlog — упорядоченный список задач).
- **Scrum Master (Скрам-мастер):** лидер-фасилитатор. Он не является техническим начальником, его задача — следить за тем, чтобы все участники строго соблюдали процессы Scrum, помогать команде спланироваться и оперативно устранять любые внешние препятствия, мешающие разработке.
- **Команда разработки (Scrum Team):** самоорганизующаяся группа специалистов (разработчики, тестеры, дизайнеры), совместно создающая инкремент продукта.

Процесс Scrum состоит из обязательных регулярных событий: Планирование спринта (выбор задач на цикл), Ежедневный Scrum (Daily Standup — 15-минутная сверка статусов), Обзор спринта (демонстрация готового инкремента заказчику) и Ретроспектива (внутренний анализ работы команды для улучшения инженерных процессов).

| Билет №29

1. Понятие спецификации языка программирования. Синтаксис языка программирования. Стиль программирования:

Качественное написание программного кода базируется на трех уровнях стандартизации:

1. **Спецификация языка:** официальный технический документ, детально описывающий стандарты, внутреннее устройство, семантику и полные возможности языка программирования. Для Python эталоном является документация, сопровождающая его эталонную реализацию CPython, а также развивающийся набор документов PEP (Python Enhancement Proposals).

2. **Синтаксис языка:** система грамматических правил, жестко определяющая, какие сочетания символов, операторов и ключевых слов формируют синтаксически корректную программу. В Python синтаксис уникален: в отличие от си-подобных языков, здесь для выделения логических блоков кода (внутри функций, циклов, условий) применяются строго отступы, а не фигурные скобки {}, а в конце инструкций не ставится точка с запятой ;.
3. **Стиль программирования:** свод рекомендаций и соглашений о правильном оформлении исходного кода, направленный на повышение читаемости программы человеком. В Python главным и непререкаемым библией стиля является документ ****PEP 8****. Он жестко регламентирует правила расстановки пробелов, максимальную длину строки (до 79 символов), порядок импорта модулей и стандарты именования переменных (snake_case) и классов (CamelCase). Чистый стиль критически важен, так как код гораздо чаще читается людьми при поддержке, чем пишется.

2. Lean методология:

Lean Software Development (Бережливая разработка ПО) — это методология управления проектами, адаптированная в ИТ-сферу из концепции бережливого производства автомобильного концерна Toyota. Центральная идея Lean заключается в жесткой оптимизации процессов разработки за счет ****тотального и непрерывного устранения любых видов потерь (Waste)****.

Потерями в индустрии программирования признается все, что тратит ресурсы команды, но не приносит прямой ценности конечному пользователю. Сюда относятся: создание избыточного функционала (о котором клиент не просил), задержки в процессах передачи задач, чрезмерная бюрократизированная документация, дефекты и баги кода (требующие времени на переделку), а также простои сотрудников.

Lean базируется на семи ключевых принципах: 1) Исключение потерь; 2) Акцент на непрерывном обучении команды; 3) Принятие критических решений как можно позже (когда накоплен максимум фактических данных); 4) Максимально быстрая доставка продукта клиенту (выпуск MVP); 5) Усиление и мотивация команды; 6) Обеспечение целостности и качества продукта; 7) Оптимизация всей системы управления в целом, а не её отдельных изолированных фрагментов.

| Билет №30

1. Разработка графического интерфейса пользователя:

Инженерия разработки графического интерфейса (GUI) — это комплексный междисциплинарный процесс, проходящий через обязательные последовательные этапы проектирования:

1. **Анализ и сбор требований:** исследование целевой аудитории приложения, формирование портретов пользователей и определение ключевых пользовательских сценариев взаимодействия.
2. **Прототипирование (Wireframing):** создание низкодетализированных монохромных набросков и каркасов экранов (в специализированных инструментах вроде Figma или Balsamiq). Цель этапа — утвердить логику переходов между окнами до этапа написания кода.
3. **Архитектурное проектирование UI:** выбор шаблона проектирования для отделения бизнес-логики от графического интерфейса. Применяются паттерны MVC (Model-View-Controller) или MVVM. В среде Kivu эта задача изящно решается декларативным разделением: логика пишется на Python, а интерфейсView — на языке KV.
4. **Реализация и программирование:** верстка компонентов, привязка стилей, цветов и логики обработки событий.

5. **Юзабилити-тестирование:** оценка удобства работы с интерфейсом на реальных пользователях, сбор метрик и устранение барьеров взаимодействия.

2. Kanban методология:

Kanban — это гибкая методология разработки, входящая в семейство Agile, которая фокусируется на полной визуализации текущего рабочего процесса команды и управлении потоком создания ценности. В отличие от фреймворка Scrum, Kanban является эволюционным, а не революционным подходом: он не вводит жестких искусственных ролей (вроде скрам-мастера) и полностью отказывается от фиксированных по времени спринтов — работа течет непрерывно.

Главным инструментом выступает ****Канбан-доска**** (физическая или виртуальная в Jira/Trello), разделенная на столбцы стадий жизненного цикла задачи: «Бэклог/Сделать», «Проектирование», «Разработка», «Тестирование», «Готово». Каждая задача оформляется в виде отдельной карточки, которая плавно мигрирует по доске слева направо по мере выполнения.

Фундаментальное правило Kanban — ****введение ограничений на незавершенную работу (WIP Limits — Work In Progress Limits)****. На каждый столбец (особенно на стадию разработки и теста) накладывается жесткий лимит (например, в колонке "Разработка" одновременно может находиться не более 3 задач). Если лимит достигнут, программисты не имеют права брать новую задачу из бэклога — они обязаны подключиться и помочь коллегам протолкнуть текущие зависшие задачи до стадии "Готово". Это позволяет мгновенно обнаруживать и устранять «узкие места» (bottlenecks) в производственном потоке команды.